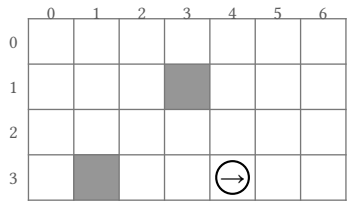
Turn Right, in Code

Name: _____ Date: _____



In one step of cleaning:

- If the square in front of the robot is open floor, it moves forward one square.
- If the square in front is a wall or furniture, it turns right a quarter turn and stays put.

Every square it leaves gets marked clean. Pictured: the room and robot built in `main`. In `grid[row][col]`, `row` is `y` and `col` is `x`.

Thursday's code, from `github.com/ucsd-cse11/lecture13`:

```
// What type goes in the 2d array?
// Use a `char`: empty is ' ', 'O', '*'
// Can be a class — all arrays are mutable anyway
record Room(char[][] grid) {}
// degrees can be 0 (up), 90 (right), 180 (down), 270 (left)
record Robot(int x, int y, int degrees) {
    String show() {
        if(degrees == 0) { return "^"; }
        if(degrees == 90) { return ">"; }
        if(degrees == 180) { return "v"; }
        if(degrees == 270) { return "<"; }
        else { return "?"; }
    }
}
class Simulation {
    final Room room;
    Robot robot;
    Simulation(Room room, Robot robot) {
        this.room = room;
        this.robot = robot;
    }
    boolean isFacingWall() {

    }
    boolean isFacingObstacle() {

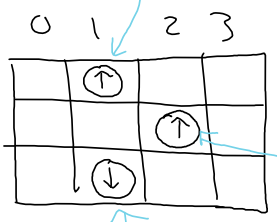
    }
    boolean isFacingWallOrObstacle() {

    }
    String show() {
        String toShow = "";
        for(int row = 0; row < room.grid.length; row += 1) {
            for(int col = 0; col < room.grid[0].length; col += 1) {
                if(col == robot.x && row == robot.y) {
                    toShow += robot.show();
                }
                else {
                    toShow += room.grid[row][col];
                }
            }
            toShow += "\n";
        }
        return toShow;
    }
}

void main() {
    Room room = new Room(new char[][]{
        { '-', '-', '-', '-', '-', '-', '-' },
        { '-', '-', '-', 'O', '-', '-', '-' },
        { '-', '-', '-', '-', '-', '-', '-' },
        { '-', 'O', '-', '-', '-', '-', '-' },
    });
    Robot r = new Robot(4, 3, 90);
    Simulation s = new Simulation(room, r);
    System.out.println(s.show());
}
```

`boolean isFacingWall()`

```
if(robot.degrees()==0) { // robot moving up, y-1
    return robot.y()==0;
} else if (robot.degrees()==90) {
    return robot.x()==room.grid()[0].length-1;
} else if (robot.degrees()==180) {
    return robot.y()==room.grid().length-1
} else {
    return robot.x()==0;
}
```



x = 1, y = 0
degrees = 0

x = 2
y = 1
degrees = 0

x = 1
y = 2
degrees = 180

room.grid().length
= 3

`boolean isFacingObstacle()`

`boolean isFacingWallOrObstacle()`

Answer Box #1

How did you represent these two Expressions?

"represent" means as Java values

$((4 + 5) * x)$

$4 + (5 * x)$

record Add(____) impl Expr

record Mul(____) impl Expr

new Mul(new Var("x"), new Add(new Num(4), new Num(5)))

this is representing $x * (4 + 5)$

new Add(new Num(4), new Mul(new Num(5), new Var("x")))

PA2 grades released resubmit due Friday Nov 13

PA1 resubmissions soon

Exam 2 grades Thu